

Project Artefact Report: Lock-In

SIT210 Embedded Systems Development, Task 11.2PCDHD

Samridh Limbu (s224412776)

Repository: <https://github.com/clupai8o0/lock-in-complete>

Demo video: https://drive.google.com/file/d/1qt07oAXUywDpyKSvX_LEvOLFmOW_XR3S/view?usp=sharing

Lock-In is a desk-side focus tracker. A Raspberry Pi 5 orchestrates an Arduino Uno sensor frontend over USB serial and a Mac webcam server over the local network. Every 75 seconds it captures a still and asks Gemini whether the person looks focused. That judgment is one input into a five-state machine that owns every decision. The model is treated as a sensor, not an oracle.

1 Project overview

1.1 Background

I built Lock-In because the tools that already exist for managing study focus all run on the very device you are trying not to get distracted by. Pomodoro timer apps, browser blockers, and screen-time dashboards are all software running on the laptop. They are easy to dismiss, and they have no way of knowing whether you actually focused or simply left a timer ticking while you scrolled your phone in your other hand.

An embedded system can sidestep that problem. A small physical box on the desk that watches the desk, makes a deterministic decision, and gives feedback through a dedicated channel of lights and a buzzer is a different kind of nudge. It is hard to ignore, hard to mute, and hard to game, because it looks at the world rather than at what the laptop chooses to report.

1.2 Existing work

The closest hobby-tier project I found was an ESP32-CAM focus buddy that pushed every frame to a local OpenCV pipeline. It worked when it worked, but it had no degradation strategy for when the camera dropped frames or the model was wrong. The gap I wanted to fill is the embedded-systems gap: real sensors, a real state machine, real fault handling, and feedback that is not just another notification on the device you are trying to ignore.

1.3 Problem statement

Build a desk-side embedded system that detects when the user is focused against distracted using physical sensors and a vision check, gives immediate physical feedback through lights and a buzzer, and persists session data for reflection. The system must keep working when any single component fails.

1.4 Requirements

2 Prototype architecture

Three communication protocols are in play. UART carries newline-delimited JSON between the Arduino and the Pi at 115200 baud. HTTP and HTTPS carry the webcam capture, the Gemini call, and the browser traffic. MQTT

carries snapshots and commands between the orchestrator and the dashboard through a local mosquitto broker. The dashboard then pushes state to the browser over Server-Sent Events, so the user interface reflects a state change in well under a second rather than waiting for a poll.

2.1 System block diagram

Figure 1 places every device and the links between them. The Pi is the only component that talks to all the others, and it is the only place that session state lives.

2.2 Flow diagram: sensor to action

Figure 2 traces a single sensor frame from the Arduino through to an actuator command and a database write, all in one pass through the orchestrator.

2.3 Finite state machine

The five states are AWAY, IDLE, FOCUS, DEGRADING, and BREAK. Every timed transition has an explicit hold time, so no transition fires on a single sample. Two global rules are not drawn to keep the figure readable. From any active state, no presence for five minutes returns the system to AWAY, and a long button press of 2.5 seconds ends the session and returns to IDLE. The complete trigger and hold-time table is in `docs/fsm.md`.

2.4 Async task topology

The orchestrator runs eight concurrent asyncio tasks that all share one `FocusFsm` instance, and the entry point adds a ninth task that writes the file-based snapshot fallback. Any task crash unwinds the orchestrator through `asyncio.wait(..., return_when=FIRST_EXCEPTION)`, which systemd then restarts.

2.5 Data model

SQLite holds one parent table and three child tables. Each child references a session, giving a clean one-to-many shape for sessions to transitions, distractions, and environment readings.

3 Source code

The repository is at <https://github.com/clupai8o0/lock-in-complete>. The submission branch is `master`.

4 Testing approach

The tests sit in three layers, chosen to mirror the three risk categories of an embedded system: logic bugs, wiring bugs, and environment failures.

4.1 Unit tests (no hardware)

Three suites run with no hardware and no network. `test_fsm.py` exercises the pure-logic FSM with an injected fake clock across 13 cases, covering every transi-

Table 1: Comparable systems and where they fall short for this use case.

System	Approach	Where it falls short
Pomodoro apps (Toggl, Forest)	Software countdown on the laptop or phone	Lives on the distraction device, no awareness of real focus
Cold Turkey, Freedom Time Timer (physical)	OS-level URL or application blocklist Mechanical visual countdown	Bypassable, no behaviour feedback, no break logic One-shot, does not react to whether you actually focused
Webcam attention monitors	Webcam machine learning on the laptop	Heavy on the laptop, no hardware actuator, no logging
ESP32-CAM study-buddy projects	DIY microcontroller plus cloud vision	Vision runs on a flaky ESP32, no state machine, no fault handling

Table 2: Functional requirements and how each is met.

#	Requirement	Implementation	Met
R1	Detect presence at the desk (motion and distance)	PIR on a hardware interrupt plus an HC-SR04 ultrasonic sensor	✓
R2	Classify whether the user is focused while at the desk	Gemini vision judge on a 75 second cadence	✓
R3	Drive a state machine with five distinct states and hysteresis	FocusFsm, covered by 13 unit tests	✓
R4	Give two feedback channels, light and audio	Three LEDs and a piezo buzzer	✓
R5	Persist session, distraction, and environment data	SQLite across four tables	✓
R6	Web dashboard for live state and history	Flask on port 8080	✓
R7	Survive disconnection of every sensor and network path	See Section 5	✓
R8	Auto-start on boot, restart on crash, recover after power loss	systemd plus an orphan-session sweep	✓
R9	Tunable from the dashboard without a restart	MQTT command topic with a file fallback	✓

Table 3: Module responsibilities.

Module	Responsibility
config.py	Env-driven Config plus derived paths
database.py	SQLite schema, thread-safe wrapper, queries
serial_reader.py	Async serial reader and writer, auto-reconnect on EOF
camera_client.py	Async HTTP client for the Mac webcam server
vision_judge.py	Gemini call plus a strict JSON parser
fsm.py	FocusFsm, pure state machine, no input or output
mqtt_bus.py	Async MQTT pub/sub on the orchestrator side
orchestrator.py	Wires modules into asyncio tasks, owns the FSM
main.py	Entry point, logging, signals, snapshot writer
dashboard/app.py	Flask, SSE stream, CSV export, command POSTs
dashboard/mqtt_bridge.py	MQTT subscriber and SSE fan-out
arduino/lock_in_arduino	Uno firmware
mac_camera/	Mac webcam Flask server

tion, every hysteresis hold, every button gesture in every legal state, and edge cases such as low-confidence vision being ignored, a null distance being treated as away from the desk, and a vision-triggered distraction not being recovered by ultrasonic alone. `test_vision_judge.py` drives the Gemini JSON parser through 25 cases covering code fences, surrounding prose, truncation, and wrong types. `test_serial_reader.py` covers auto-reconnect on EOF, queue-overflow drops, and parse errors across 9 cases. A fourth suite, `test_mqtt_integration.py`, runs 4 live pub/sub round-trips between the orchestrator and the dashboard and skips automatically when no broker is listening on 127.0.0.1:1883.

```
cd pi
python -m unittest discover -p 'test_*.py'
```

4.2 Integration tests (hardware on the bench)

These are manual but scripted. Each row was run at least once before submission.

4.3 Fault-injection tests

Each failure path in Section 5 was exercised by removing the dependency and watching `journalctl` for the expected log line and the dashboard for the expected degraded state.

5 Fault tolerance

Seven classes of failure are handled. The principle in each case is the same: detect, contain, degrade, then recover.

The cross-cutting guards are:

- **systemd**: `Restart=always`, `RestartSec=5`, and a real `Type=notify` watchdog with `WatchdogSec=120` paired with a 30 second `WATCHDOG=1` feed. The dashboard is a separate unit, so neither service can take the other down.
- **asyncio supervisor**: `return_when=FIRST_EXCEPTION` in `Orchestrator.run`. A task that enters a bad state should not silently keep running.
- **SQLite WAL with synchronous NORMAL**: a crash mid-write at worst loses the last in-flight transaction.
- **Atomic snapshot write**: the snapshot writer writes a temporary file and renames it into place, so the dashboard never reads a half-written file.
- **Bounded queue**: the serial queue of 128 entries drops the oldest rather than back-pressuring the producer.
- **Strict input validation**: vision judgments below 0.6 confidence are ignored, non-JSON serial and MQTT messages are dropped, and the dashboard endpoints allow-list actions and settings keys.

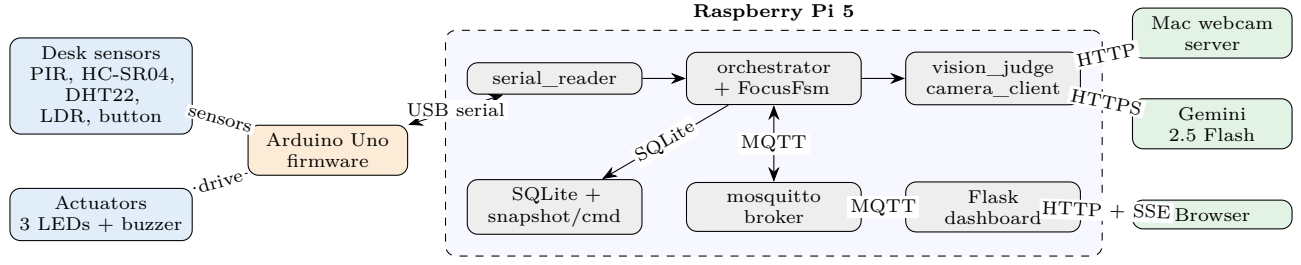


Figure 1: Hardware and network architecture. UART links the Arduino and the Pi, HTTP and HTTPS reach the Mac webcam and Gemini, and MQTT plus SSE link the orchestrator, the broker, the dashboard, and the browser.

Table 4: Bench integration checks.

Test	Method	Pass criterion
Sensor frames arrive at 1 Hz	Watch <code>journalctl</code> for <code>frame #N</code> lines	Less than 1.2 s between consecutive frames
Button gestures detected	Press single, double, long	The right action is logged each time
Transitions reach the Arduino	Force a transition, check the LED	LED pattern matches the state mapping
Vision call round-trips	Trigger a vision cycle	Judgment line in the log and the dashboard updates
Dashboard live updates	Open the dashboard, force a transition	State badge changes within about two seconds
CSV export	Click Download CSV (30 days)	File opens in a spreadsheet with no errors
Settings applied live	Change pomodoro length in the dashboard	Next pomodoro uses the new value with no restart

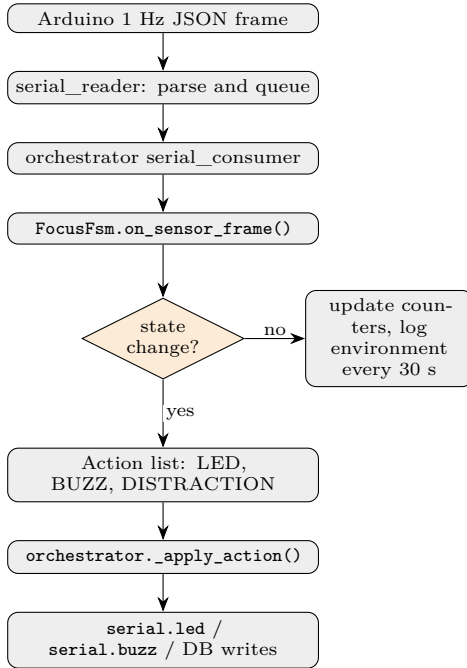


Figure 2: A sensor frame travels from the Arduino to an actuator and a database write in one pass.

5.1 Fault-handling code

The four listings below are the load-bearing parts of that table. Each is the actual code from the repository, trimmed only for length.

F1: serial reconnect and bounded queue. The reader treats EOF (Arduino unplugged) as a normal event: it logs, sleeps, and reconnects. The queue is bounded, so a stalled consumer can never grow memory without bound; the newest sensor frame always wins.

```
async def run(self) → None:
    while not self._stop.is_set():
        try:
            reader, writer = await serial_asyncio \
                .open_serial_connection(url=self.port,
                                       baudrate=self.baud)

            self._connected.set()
            await self._read_loop(reader) # returns on EOF
        except Exception as e:
            log.warning("serial connect failed: %s", e)
        finally:
            self._connected.clear()
            if not self._stop.is_set():
                await asyncio.sleep(self.reconnect_delay_s)

# inside _read_loop, on a full queue:
try:
    self.queue.put_nowait(msg)
except asyncio.QueueFull:
    self.queue.get_nowait() # drop oldest
    self.queue.put_nowait(msg) # keep newest
```

F2 to F4: vision degrades to “no opinion”. Every failure mode of the vision path, namely camera offline, network error, a 20 s timeout, or unparseable JSON, collapses to the same return value of `None`. The orchestrator then simply skips the cycle and the FSM keeps running on sensors alone.

```
async def judge(self, jpeg, *, timeout_s=20.0):
    loop = asyncio.get_running_loop()
    try:
        return await asyncio.wait_for(
            loop.run_in_executor(None, self._judge_sync, jpeg),
            timeout=timeout_s)
    except asyncio.TimeoutError:
        self.last_error = "timeout"
        return None # treated as "no opinion this cycle"

# _judge_sync swallows every API/parse error the same way:
except Exception as e:
    self.last_error = f"{type(e).__name__}: {e}"
    return None
```

F5: power-loss recovery on boot. A session row with a `NULL end_time` can only mean the Pi died mid-session. The startup sweep seals every such row with a note, so the

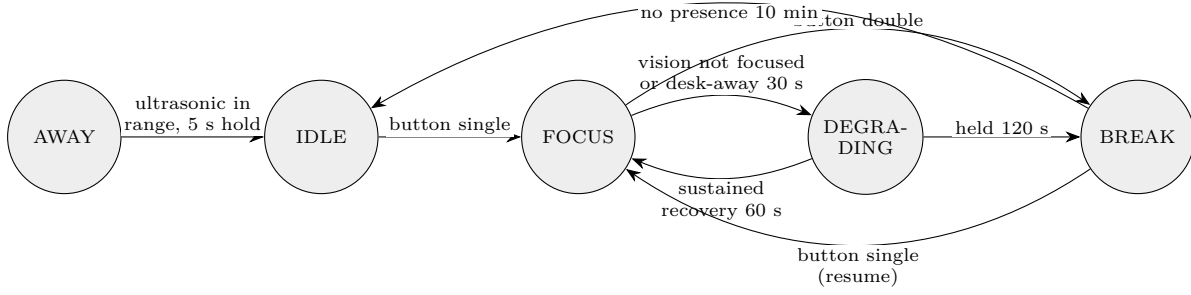


Figure 3: State diagram. Hysteresis on every edge prevents oscillation when a single reading is noisy.

Table 5: Failure handling. Each path was verified by fault injection.

#	Failure	Detection	Degradation	Recovery
F1	Arduino unplugged	<code>readline()</code> returns EOF	FSM holds state, frames stop arriving	5 s reconnect loop, frames resume
F2	Mac webcam offline	aiiohttp timeout or client error	Vision cycle returns None, sensor logic continues	Next cycle reopens the connection
F3	Gemini returns bad JSON	Parser returns None	One cycle skipped, no database write	Retry after 75 s
F4	Gemini timeout or slow	<code>asyncio.wait_for</code> at 20 s cancels	Cycle skipped	Retry next cycle
F5	Pi power loss mid-session	Open session row has <code>end_time</code> NULL at boot	Startup sweep seals it with a note	Next run continues normally
F6	MQTT broker down	Publish or connect failure	Dashboard and orchestrator fall back to <code>snapshot.json</code> and <code>cmd.json</code>	Bus reconnects every 5 s
F7	Orchestrator hangs	systemd watchdog, three missed 30 s feeds	Process killed	Restarted after 120 s

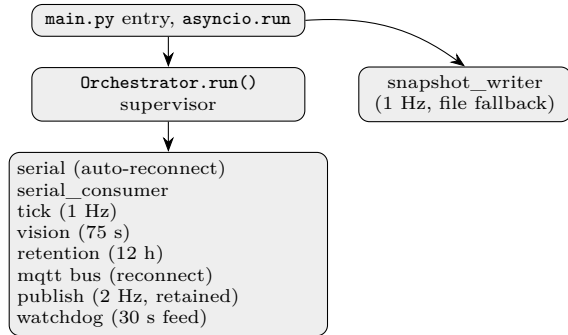


Figure 4: Concurrency layout. Eight orchestrator tasks plus the snapshot writer.

data is never left in a half-open state.

```
def mark_orphan_sessions_incomplete(self) → None:
    with self._cursor() as cur:
        cur.execute(
            "UPDATE sessions SET end_time = ?, "
            "notes = COALESCE(notes, '') "
            "|| ' [incomplete: pi restarted]' "
            "WHERE end_time IS NULL", (_now_iso(),))
```

F7: supervisor and systemd watchdog. Any task that crashes wins the `FIRST_COMPLETED` race, so the supervisor logs it and unwinds cleanly rather than limping on in a half-dead state, and systemd then restarts the unit. A separate task feeds the hardware-style watchdog, so a hang, not just a crash, is also caught.

```
done, pending = await asyncio.wait(
    [*tasks, stop_task], return_when=asyncio.FIRST_COMPLETED)
for t in done:
    if t is not stop_task and t.exception():
        log.exception("task crashed: %s", t.get_name(),
            exc_info=t.exception())

async def _watchdog_loop(self) → None:      # WatchdogSec=120
    while not self._stop.is_set():
        sd_notify.watchdog()                # WATCHDOG=1
        await self._interruptible_sleep(30.0) # 3 missed → restart
```

6 Design principles

The system is built around six principles. Each is also a constraint that forced one or more concrete decisions.

Separation of policy and mechanism. The FSM in `pi/fsm.py` is pure logic with no input, output, threading, or globals. It receives events and returns `Action` records, and the orchestrator applies them. This makes the FSM testable in microseconds with a fake clock, and it means a wiring bug can never cause a state-machine bug.

Fail-open, not fail-closed. Every external dependency, namely the Arduino, the camera, and Gemini, is optional from the orchestrator's point of view. A failure returns `None` and the loop continues. The opposite design, refusing to operate when any sensor is missing, was rejected because the goal is behavioural feedback during a study session. A system that bricks itself when the network drops would defeat the purpose.

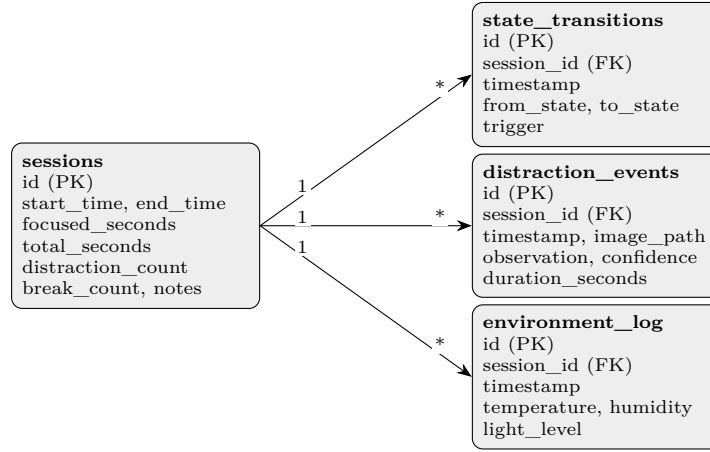


Figure 5: Entity relationships. One session owns many transitions, distractions, and environment rows.

Determinism through hysteresis. Every timed transition has an explicit hold-time constant, so nothing fires on a single sample. An earlier draft oscillated between **FOCUS** and **DEGRADING** every few seconds when I leaned back to think. The current design requires 60 seconds of confirmed focus before recovery and 30 seconds of confirmed absence before degrading, with a 0.6 confidence floor on vision.

Defence in depth. Two feedback channels mean losing the LED still leaves the buzzer. Two inter-process paths, MQTT as the primary and the snapshot and command files as the fallback, mean the dashboard keeps working when the broker is down. Two restart safety nets, the asyncio supervisor and **Restart=always**, mean a single layer of crash handling is never the only line of defence.

Responsiveness through decoupled messaging. The orchestrator and the dashboard never call each other directly. The orchestrator publishes a retained snapshot to MQTT twice a second, the dashboard bridge caches it and fans it out to browsers over Server-Sent Events, and a freshly connected browser sees the current state at once because the snapshot is retained. A state change reaches the screen in well under a second.

Observability over instrumentation. The system logs at INFO whenever state changes and at DEBUG for every sensor frame. The dashboard surfaces live system status, namely Arduino connected, camera online, and vision enabled, so a fault is visible at a glance. Distraction snapshots are kept as JPEGs until the retention sweep purges them, which gives a forensic record of what the vision model actually saw.

A meta-principle sits behind all of these. Everything that runs on the Pi runs locally. Only the Gemini call leaves the device, and only after a graceful skip path is wired in for when that call fails.

6.1 Evaluation criteria

These principles map directly onto the four extra evaluation criteria this project targets. Each is backed by a concrete mechanism in the code, shown below.

Robustness. Covered by the fault-handling code in Section 5: every external dependency returns **None** instead of raising, the supervisor restarts on any task crash, and the watchdog catches hangs. The seven fault paths in Section 5 were each verified by fault injection.

Responsiveness. The orchestrator publishes a *retained* snapshot to MQTT at 2 Hz; the dashboard fans it out to browsers over Server-Sent Events. Retain means a newly connected browser sees current state immediately instead of waiting for the next tick, and a state change reaches the screen in well under a second.

```

async def _publish_loop(self) → None:
    while not self._stop.is_set():
        await self._interruptible_sleep(0.5)           # 2 Hz
        await self.bus.publish(self._topic_snapshot,
                               self.snapshot(), retain=True)

```

Real-time behaviour. Two loops give bounded latency at two timescales. The Pi runs a fixed 1 Hz control tick, and the Arduino handles its time-critical inputs (PIR and button edges) in hardware ISRs rather than by polling, so an event is captured the instant it happens regardless of what the main loop is doing.

```

async def _tick_loop(self) → None:           # Pi: fixed 1 Hz
    while not self._stop.is_set():
        await asyncio.sleep(1.0)
        actions = self.fsm.tick()
        await self._apply_actions(actions)

```

```

// Arduino: capture edges in interrupt handlers
attachInterrupt(digitalPinToInterrupt(PIN_PIR),
                isrPir, RISING);
attachInterrupt(digitalPinToInterrupt(PIN_BUTTON),
                isrButton, CHANGE);

void isrButton() { // runs the instant the pin changes
    buttonLevel = digitalRead(PIN_BUTTON);
    buttonEdgeMs = millis();
    buttonChanged = true; // volatile flag for the main loop
}

```

Concurrency. Eight asyncio tasks share one **FocusFsm** instance on a single thread, so no locks are needed, because the FSM is only ever touched from the event loop. They are launched together and supervised as a group.


```
tasks = [
    asyncio.create_task(self.serial.run(), name="serial"),
    asyncio.create_task(self._serial_consumer(), name="
        serial_consumer"),
    asyncio.create_task(self._tick_loop(), name="tick"),
    asyncio.create_task(self._vision_loop(), name="vision"),
    asyncio.create_task(self._retention_loop(), name="retention"),

    asyncio.create_task(self.bus.run(), name="mqtt"),
    asyncio.create_task(self._publish_loop(), name="publish"),
    asyncio.create_task(self._watchdog_loop(), name="watchdog"),
]
```

7 User manual

The full end-to-end install is in docs/setup.md. A condensed version follows.

7.1 Bill of materials

Table 6: Parts list.

Part	Qty	Notes
Raspberry Pi 5 (8 GB)	1	Pi 4B also works
Arduino Uno R3	1	Or a CH340 clone
HC-SR501 PIR	1	Set the retrigger jumper to H
HC-SR04 ultrasonic	1	5 V powered
DHT22 / AM2302	1	10k pull-up if not a breakout
LDR plus 10k resistor	1	For the voltage divider
LEDs red, yellow, green	3	With 220 ohm resistors
Push button (momentary)	1	Active-low via internal pull-up
Passive piezo buzzer	1	Must accept tone()
Mac or laptop with webcam	1	On the same network as the Pi
Breadboard and jumpers	–	

7.2 Installation

On the Pi, install the runtime and the broker, then the Python dependencies.

```
sudo apt update
sudo apt install -y python3-venv python3-pip git \
    mosquito mosquitto-clients
sudo systemctl enable --now mosquitto
sudo usermod -a -G dialout "$USER" # re-login after

git clone https://github.com/clupai800/lock-in-complete ~/lock-in
cd ~/lock-in/pi
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

cp .env.example .env
$EDITOR .env # set GEMINI_API_KEY and CAMERA_URL
```

Flash the Arduino with arduino/lock_in_arduino/lock_in_arduino.ino using Arduino IDE 2.x. The only library dependency is the DHT sensor library and its Adafruit Unified Sensor dependency. On the Mac, start the camera server.

```
cd mac_camera
pip install -r requirements.txt
python mac_camera_server.py --port 8081
```

7.3 Run

```
cd ~/lock-in/pi
./run.sh
```

Open <http://<pi-ip>:8080/>. To install as boot services:

```
sudo cp systemd/lockin-*.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now \
    lockin-orchestrator lockin-dashboard
sudo journalctl -fu lockin-orchestrator
```

7.4 Daily use

Table 7: Button gestures and presence behaviour.

Gesture	What happens
Sit at the desk	AWAY to IDLE after 5 s, yellow LED on
Single press	IDLE to FOCUS, green LED, session starts
Double press	FOCUS to BREAK, yellow blink
Single press in BREAK	BREAK to FOCUS, resume
Long press (2.5 s)	End the session, return to IDLE
Leave the desk	After 30 s to DEGRADING, after 5 min to AWAY

The dashboard has four tabs. **Today** shows the live state, a pomodoro ring, stat cards, and the latest observation. **History** shows the last seven days by day and per session with a CSV export. **Distractions** is a gallery of snapshots from the last seven days. **Settings** adjusts the pomodoro length, the hold times, and the target pomodoro count.

8 Demonstration video

The demonstration video is available at https://drive.google.com/file/d/1qt07oAXUyWdPyKSvX_LEvOLFmOW_XR3S/view?usp=sharing. The recorded video covers the four required topics, namely the purpose of the system, the hardware and network architecture, the overall operation, and the fault tolerance with its evaluation criteria, in roughly that order.

9 Conclusion

9.1 Reflection

The single biggest lesson was that vision is unreliable enough that the state machine has to be the source of truth, not the model. The first working version polled Gemini every 30 seconds and trusted the answer directly. It bounced between FOCUS and DEGRADING constantly, because a single bad frame, a hand over the face or a head briefly down to read something on paper, would tip the state. Adding the hysteresis layer, 60 seconds of confirmed recovery, 30 seconds of confirmed absence, and a 0.6 confidence floor, turned the system from frustrating into genuinely useful. The model became one input among several rather than the decision maker.

The second lesson was that failure planning is a design input, not an afterthought. When I built each module I asked what it returns when its dependency is missing before I wrote the happy path. That is why `camera_client.capture()` returns None rather than raising, why `vision_judge.judge()` swallows every exception and logs once, and why the orchestrator never assumes a frame will arrive on schedule. The result is a system I

can intentionally break in front of a marker, and it just degrades with no crashes, no half-states, and no lost data.

A third lesson came during the move to MQTT. The original dashboard polled a snapshot file once a second, which felt sluggish. Replacing that with a retained MQTT snapshot and Server-Sent Events dropped the visible latency to well under a second while keeping the file path as an offline fallback, so the improvement cost nothing in robustness.

9.2 If I had a second chance

I would do three things differently. First, I would start with the FSM and its unit tests, not with the wiring. I lost time chasing intermittent FSM bugs that turned out to be implicit timing assumptions. Second, I would invest earlier in a local vision model such as Moondream2 through Ollama as the primary path and keep Gemini as a fallback, rather than the other way around. Gemini's latency floor of around two to three seconds per call is the main reason the vision cadence has to be 75 seconds, and a local model at roughly half a second would let me poll every 15 to 20 seconds and catch shorter distractions. Third, I would design the dashboard mobile-first, because the stat cards work well on a phone but the settings forms are cramped on a small screen.

Overall the project delivered on every requirement from R1 to R9 and on the extension goals of fault tolerance, design principles, live-tunable settings, and decoupled MQTT messaging. The core is around 2,400 lines of Python plus roughly 800 lines of tests and the Arduino sketch, and every module is testable in isolation. That feels like the right shape for an embedded prototype: small, observable, and intentionally boring where it matters.